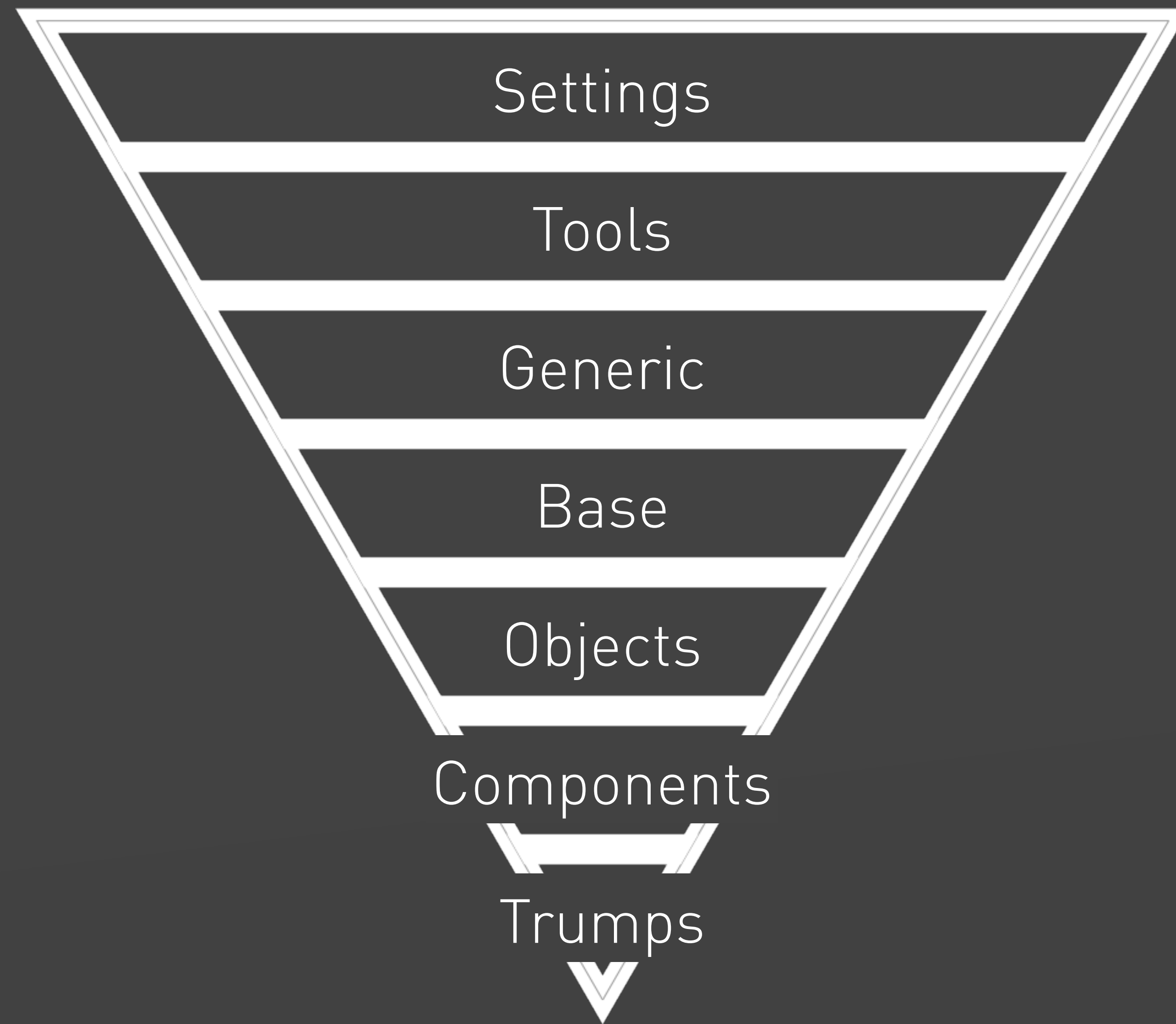




Default layers

Settings: Global variables, config switches.

Tools: Default mixins and functions.

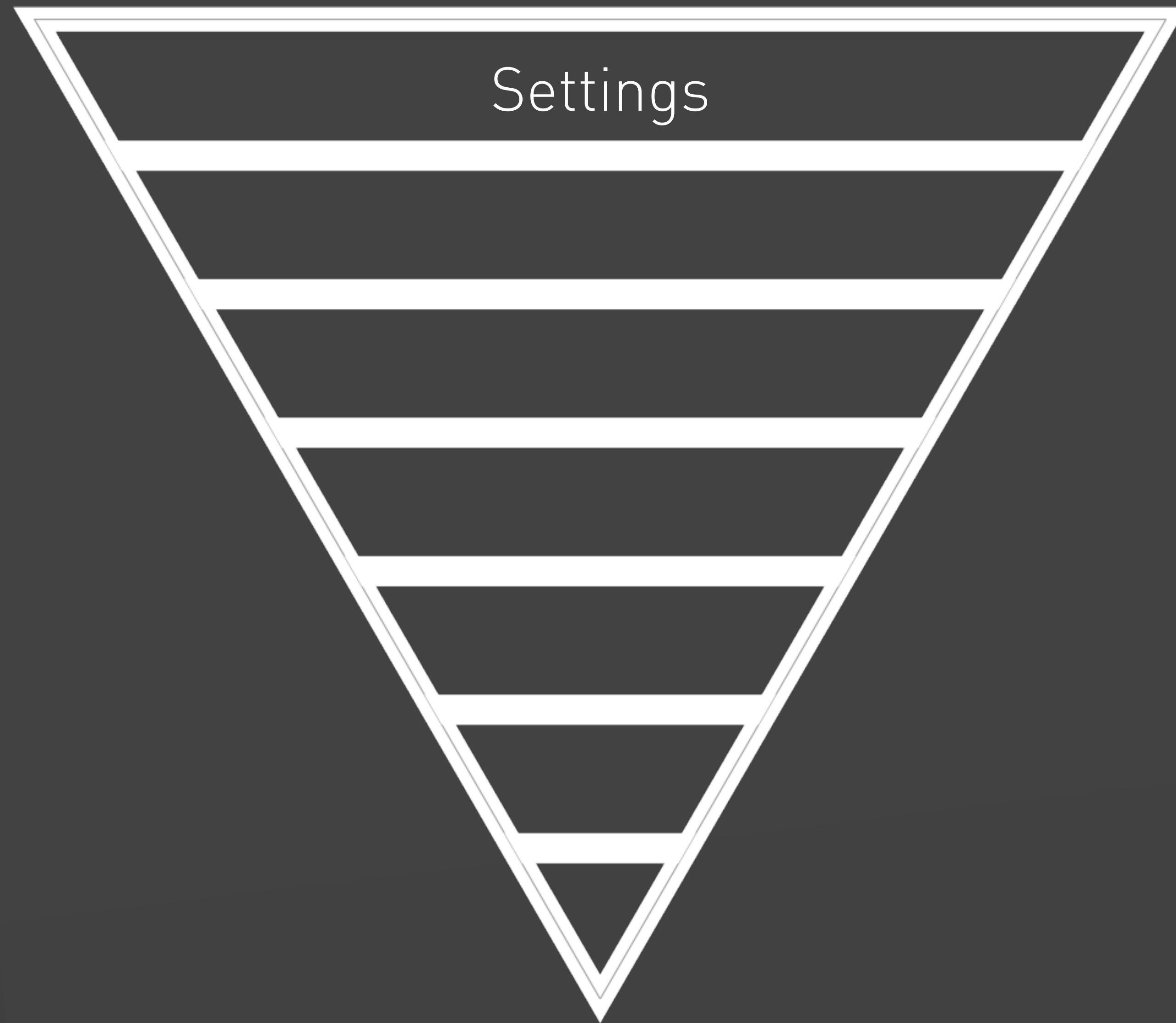
Generic: Ground-zero styles (Normalize.css, resets, box-sizing).

Base: Unclassed HTML elements (type selectors).

Objects: Cosmetic-free design patterns.

Components: Designed components, chunks of UI.

Trumps: Helpers and overrides.

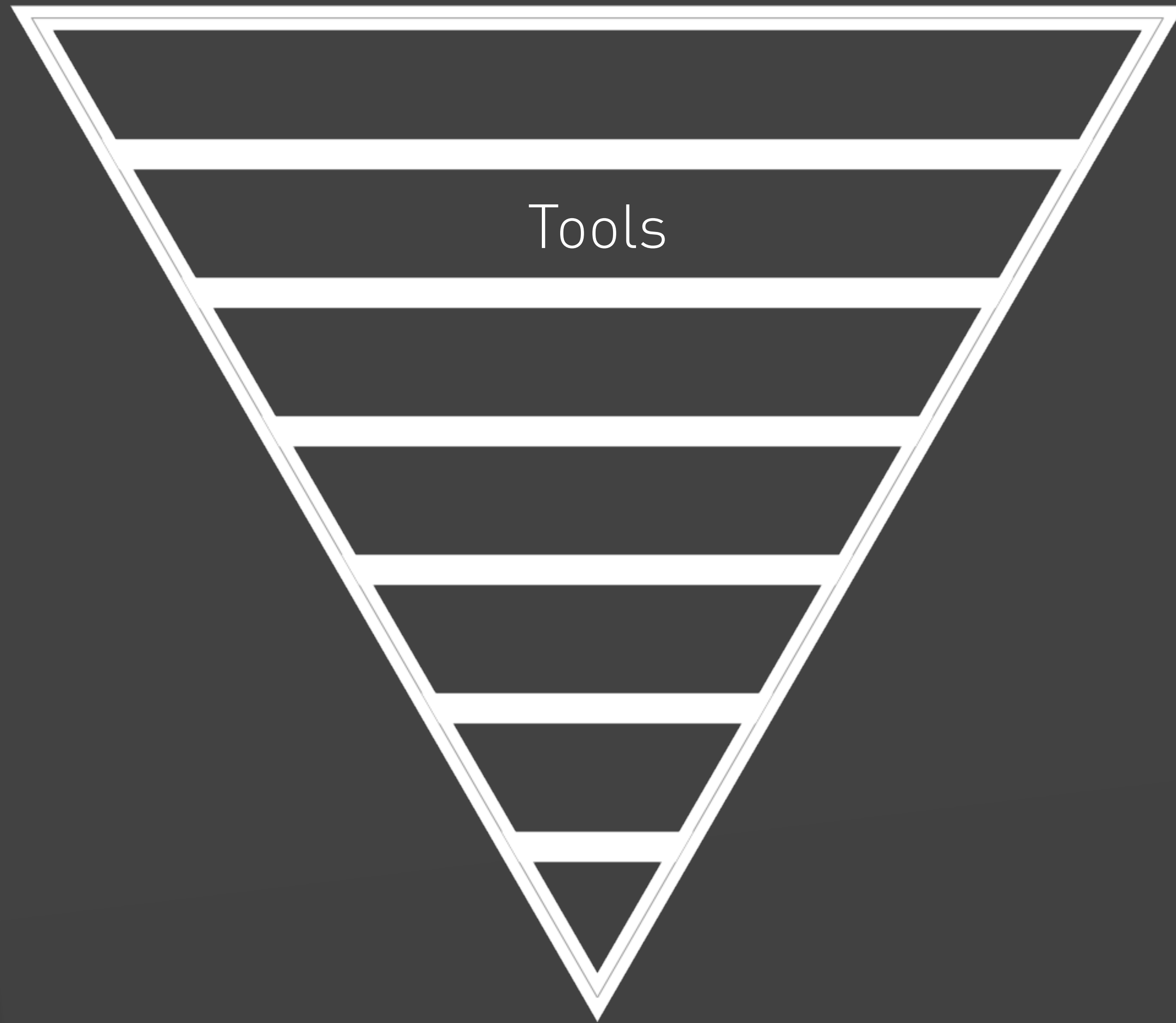


Globally-available settings.

Config switches.

Brand colours, etc.

```
$color-ui:      #BADA55;  
$spacing-unit:  10px;
```

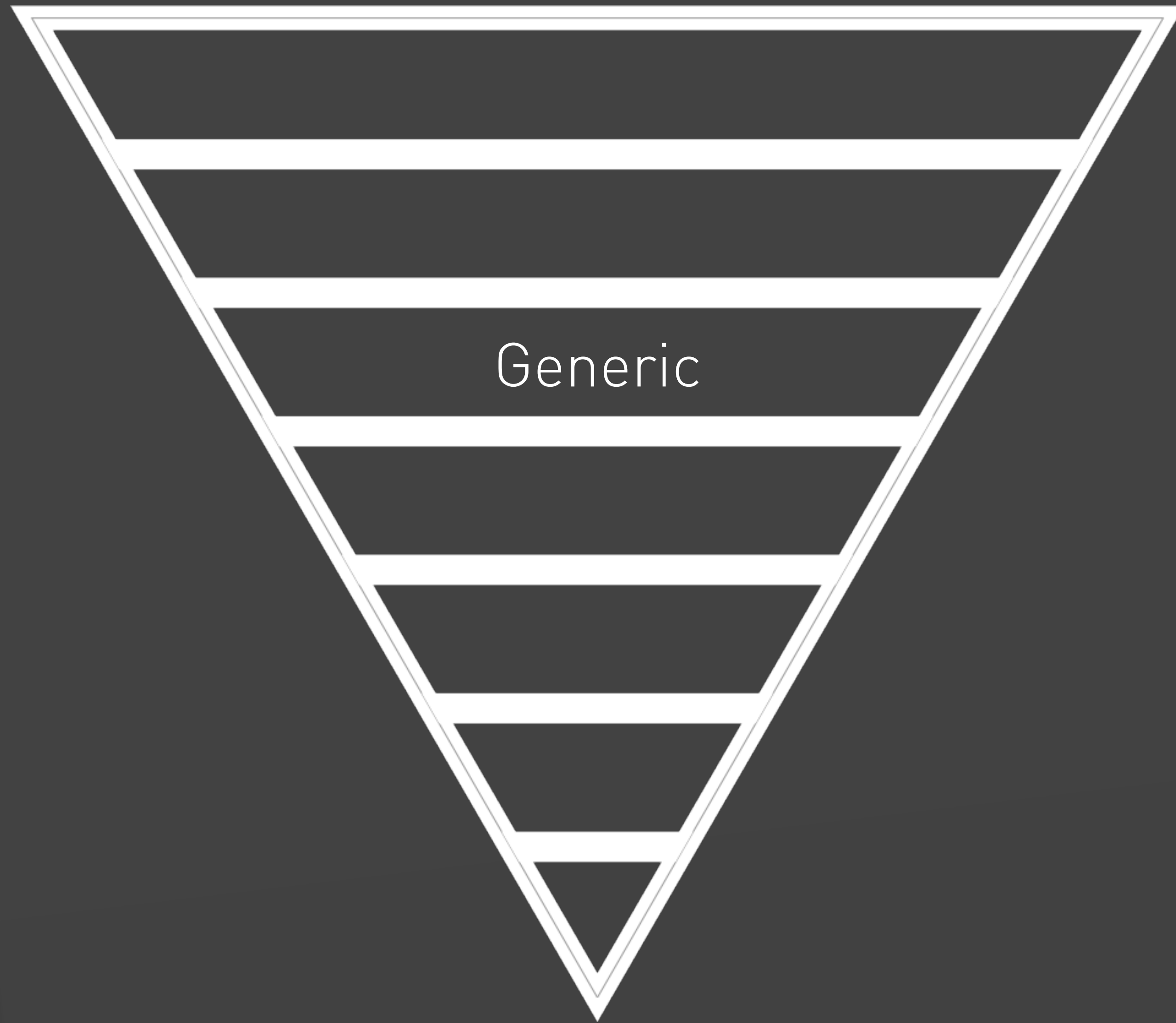


Globally-available tools.

Public mixins.

Helper functions.

```
@mixin font-brand() {  
    font-family: "UI Font", sans-serif;  
    font-weight: 400;  
}
```

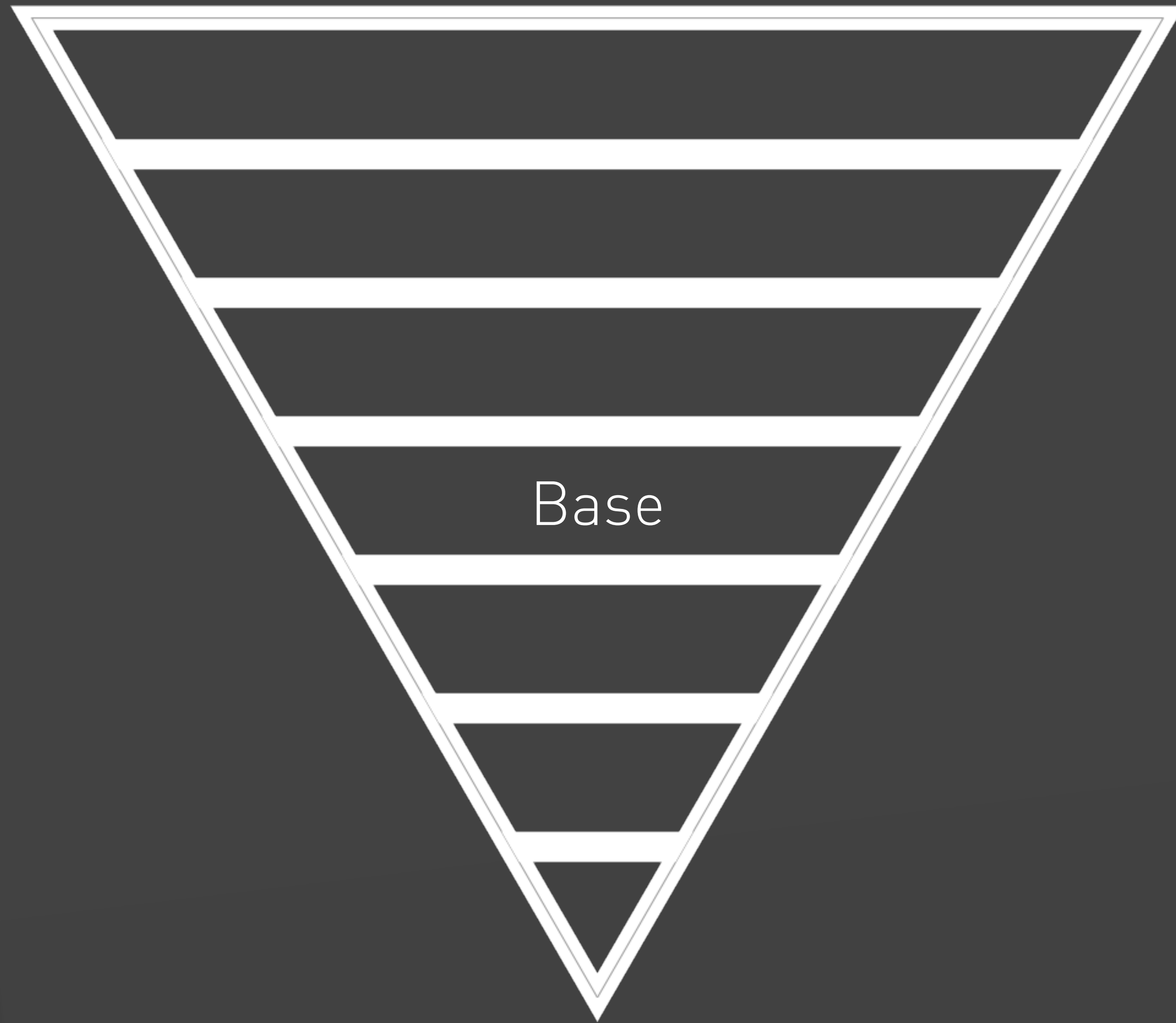


Ground zero styles.

Low-specificity, far-reaching.

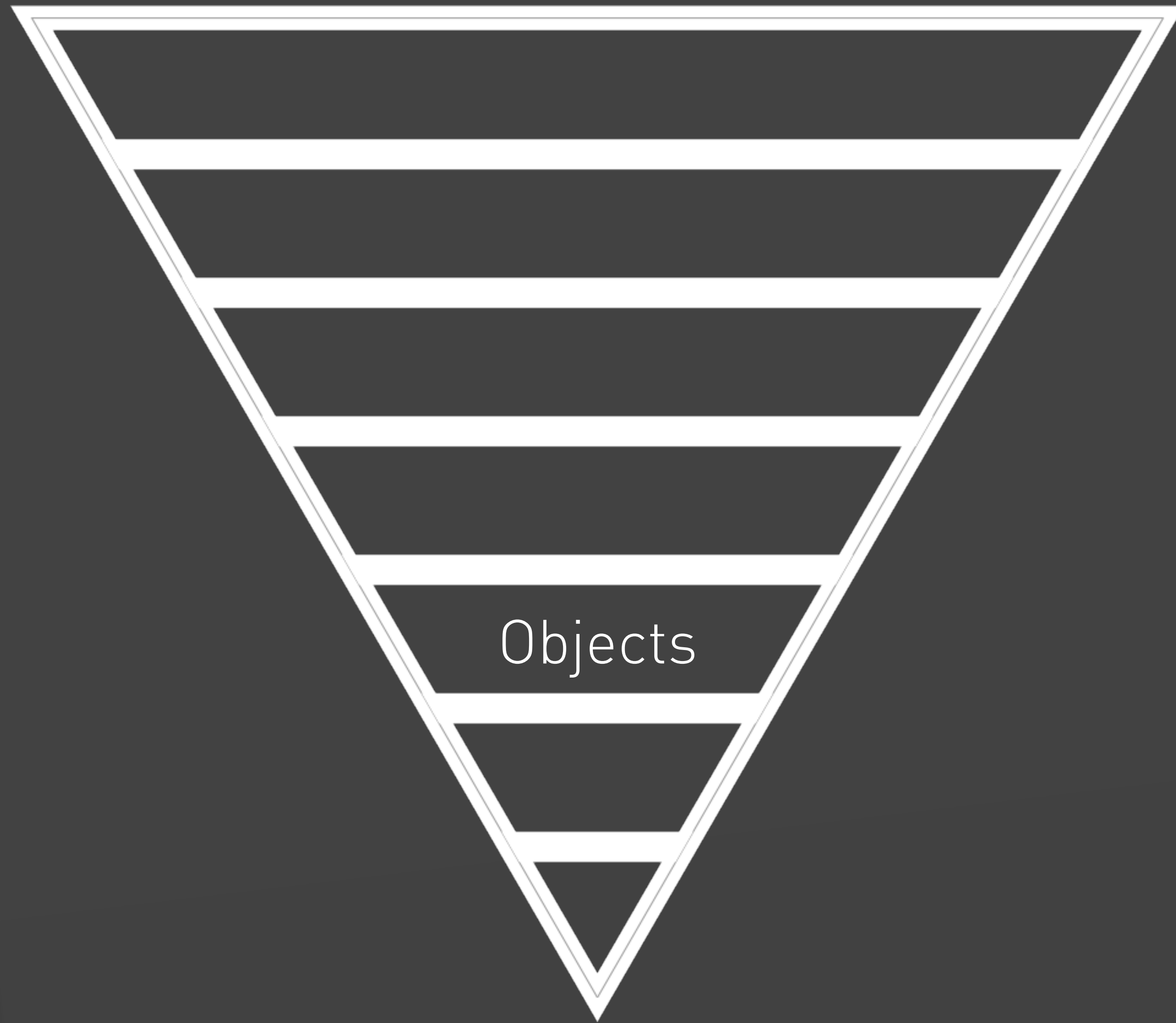
Resets, Normalize.css, etc.

```
* {  
  -webkit-box-sizing: border-box;  
  -moz-box-sizing: border-box;  
  box-sizing: border-box;  
}
```



Unclassed HTML elements.
H1–H6, basic links, lists, etc.
Last layer we see type
selectors (e.g. `a {}`,
`blockquote {}`).

```
ul {  
    list-style: square outside;  
}
```



OOCSS.

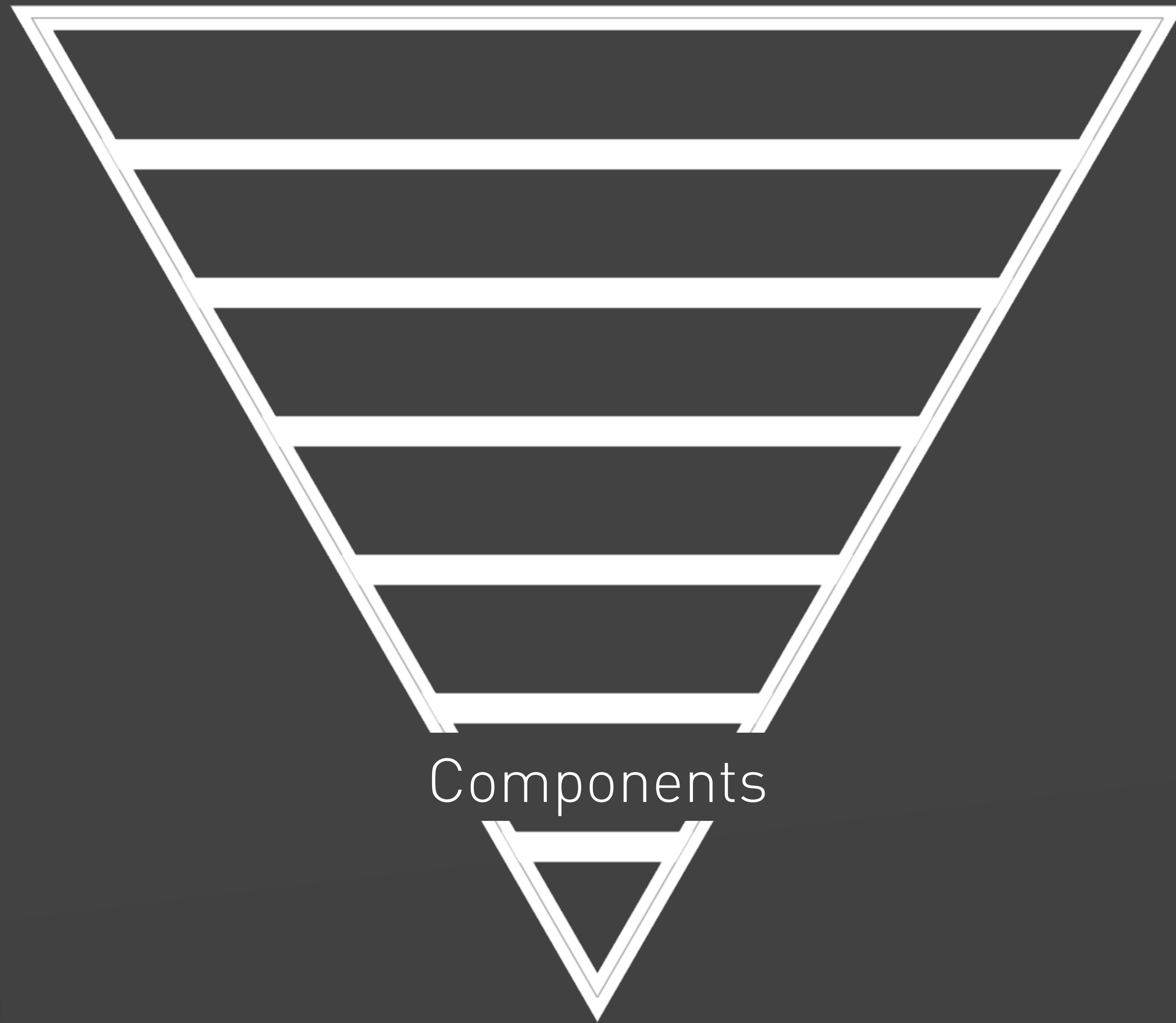
Design patterns.

No cosmetics.

Begin using classes
exclusively.

Agnostically named (e.g. `.ui-
list {}`).

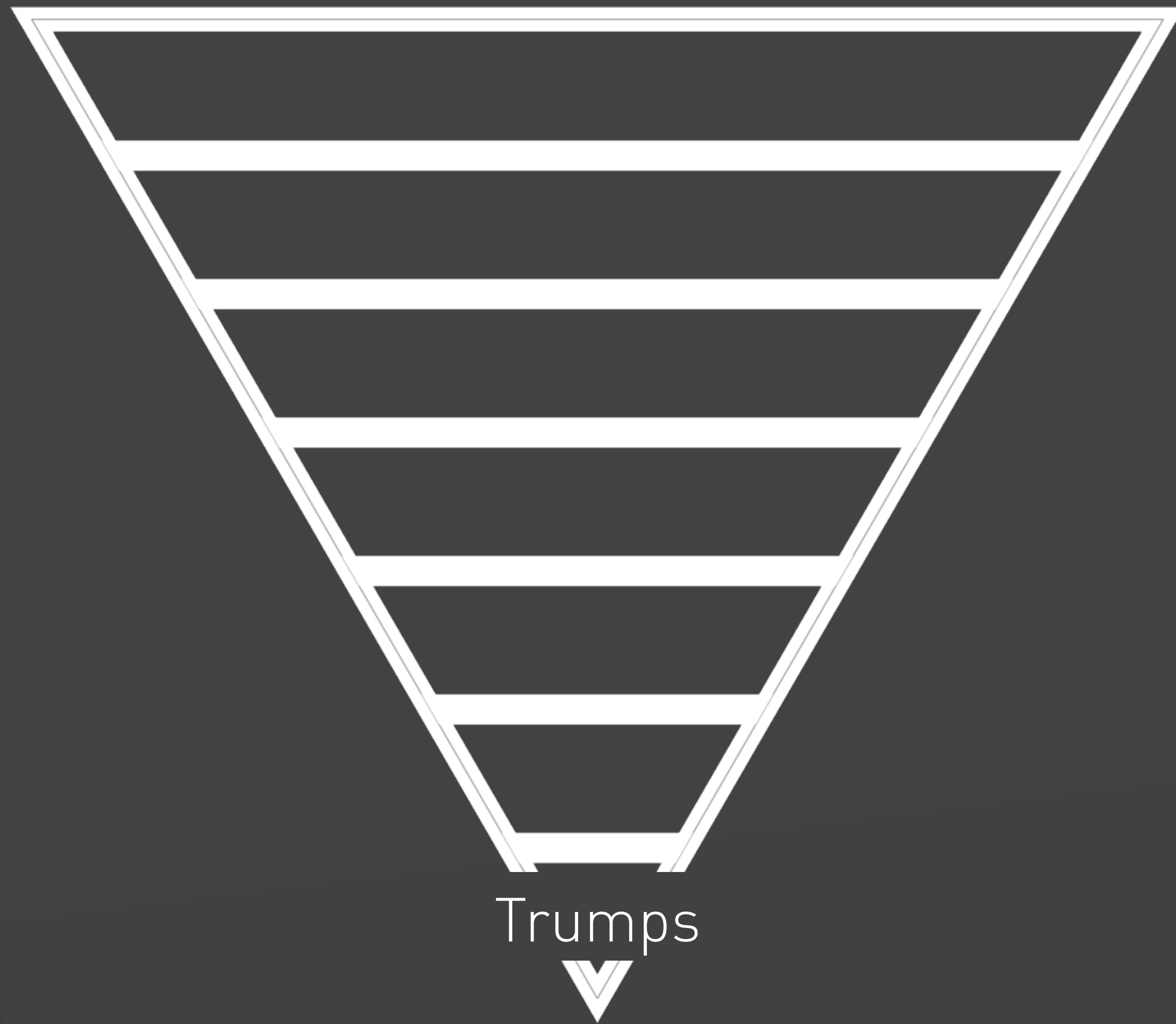
```
.ui-list {  
    margin: 0;  
    padding: 0;  
    list-style: none;  
}  
  
.ui-list__item {  
    padding: $spacing-unit;  
}
```



Designed pieces of UI.
Still only using classes.
More explicitly named
(e.g. `.products-list {}`).

```
.products-list {  
    @include font-brand();  
    border-top: 1px solid $color-ui;  
}
```

```
.products-list__item {  
    border-bottom: 1px solid $color-ui;  
}
```



Overrides, helpers, utilities.

Only affect one piece of the DOM at a time.

Usually carry !important.



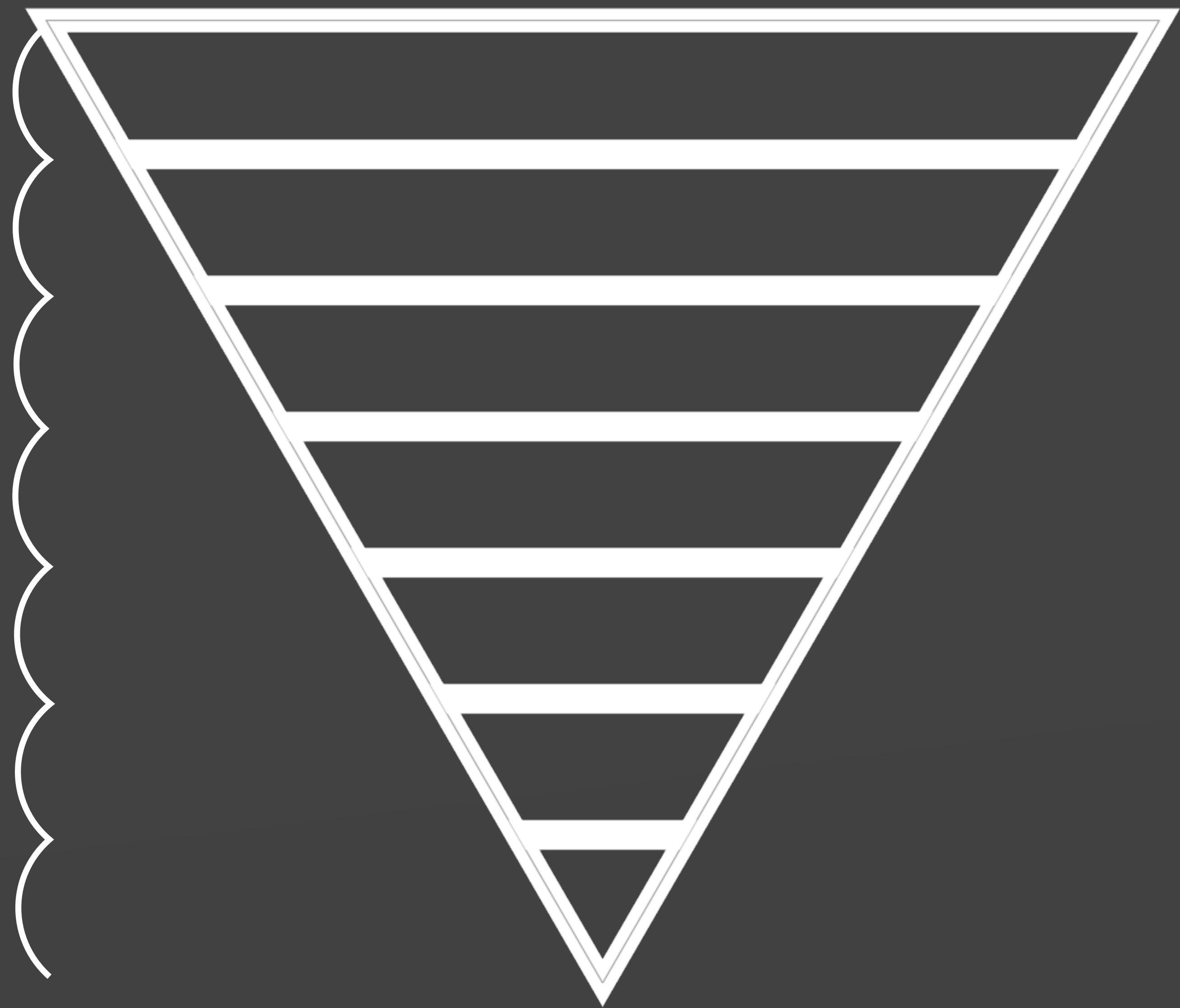
```
.one-half {  
    width: 50% !important;  
}
```

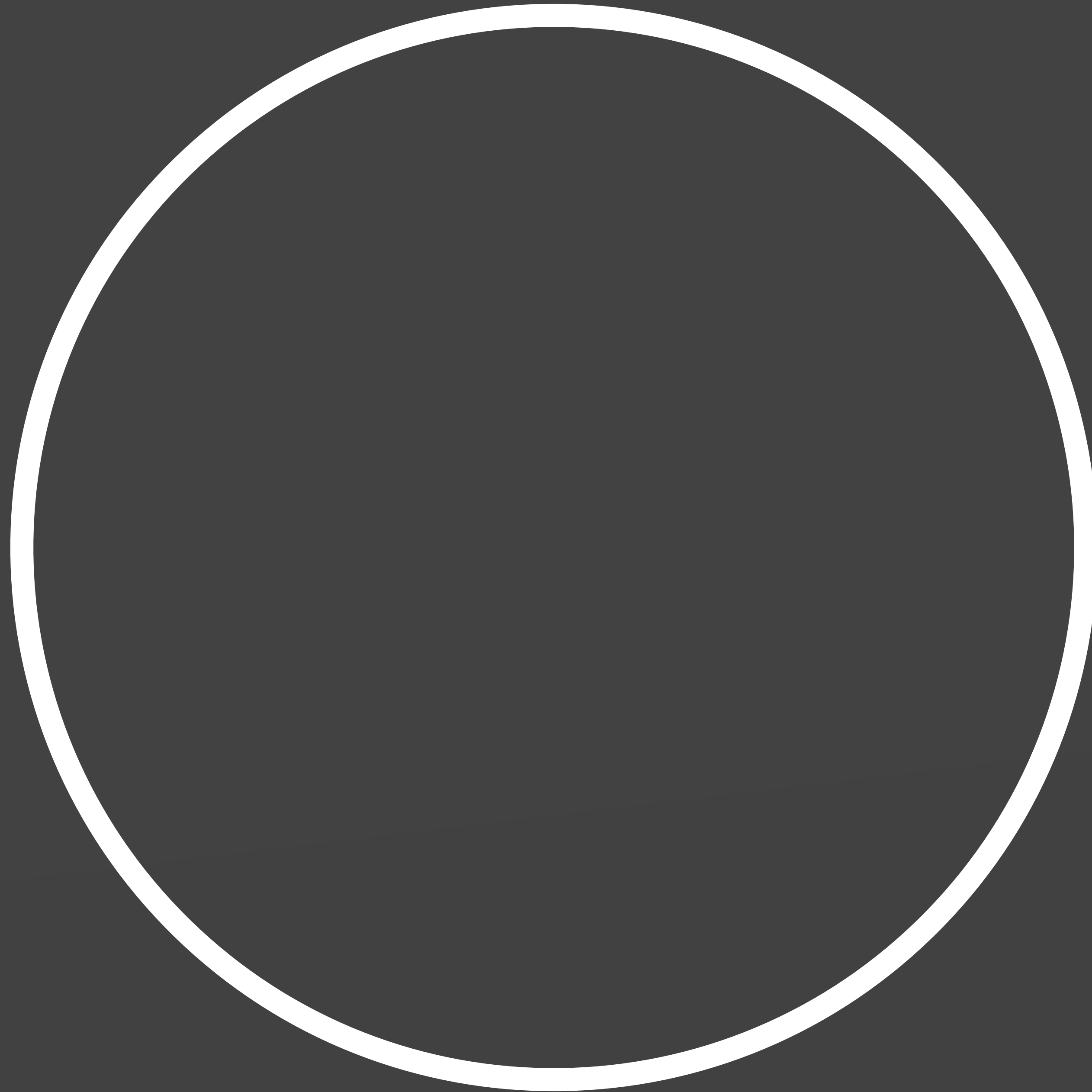
You should notice...

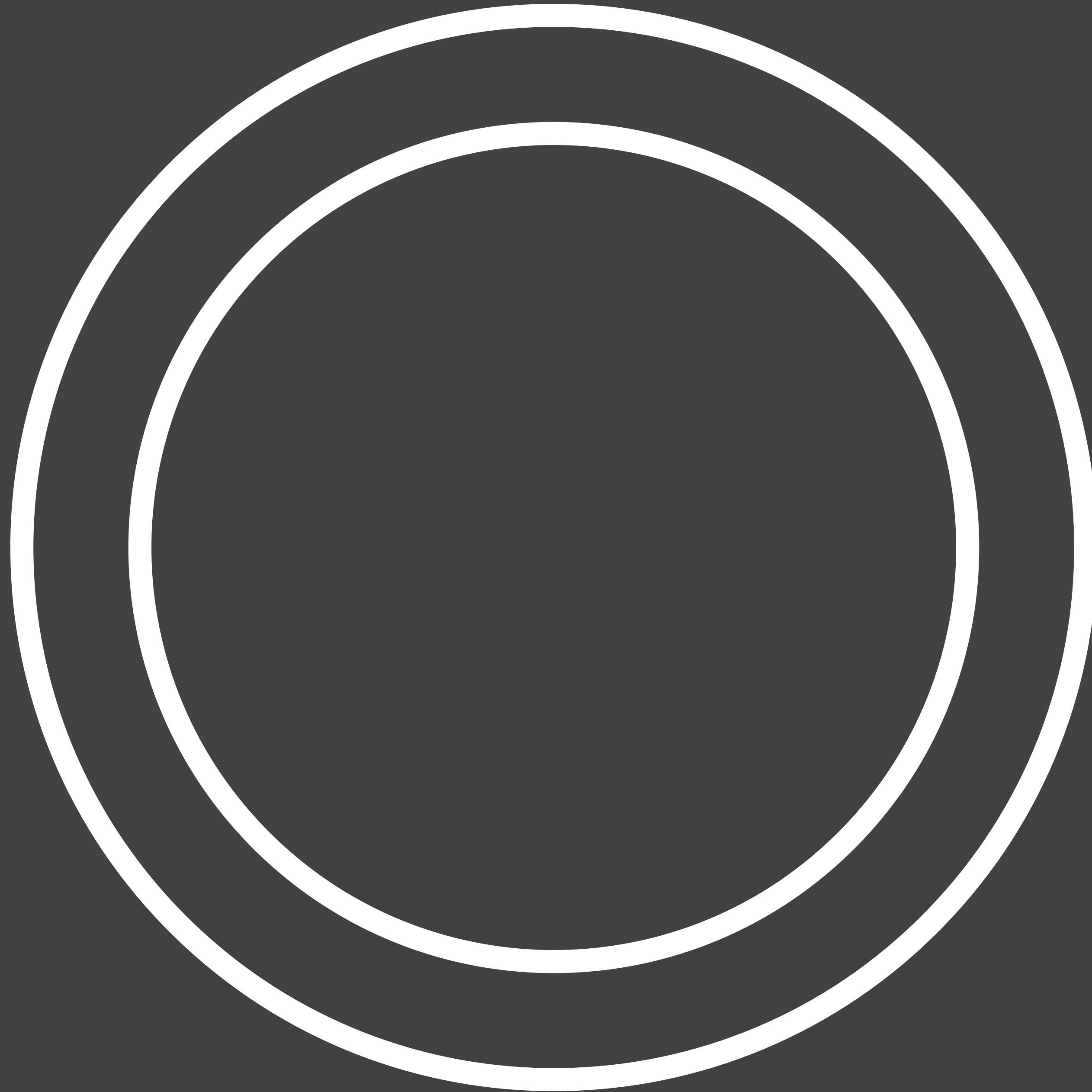
Specificity **slowly increases** layer-by-layer.

We affect **smaller and smaller** bits of the DOM at a time.

Progressively adding styles; **never undoing**.

















ITCSS...

Manages source order.

Filters **explicitness**.

Tames the cascade.

Sanitises inheritance.

Each layer is a **pass over the DOM**.

How sculptors work

Blast some rock out of a quarry.

Cut it down into a **large block**.

Rough it into a **general** shape.

Begin adding **features**.

Add fine **details**.

Each stage is **more detailed and explicit** than the last one.

Outcomes

Outcomes

Everything has a **place to live**.

People know where to look to find types of rule.

A **sane** source order.

Reduced waste/redundancy.

Increased **scalability**.

The **Specificity Wars** are over!

Scaling ITCSS

Scaling ITCSS

We can **scale our CSS** much more easily.

But we can also **scale the architecture itself!**

Scaling our CSS

No longer **the end of a stylesheet** to worry about.

Add things **into the relevant layers** (usually the last ones).

Things **never get more complicated**, only bigger.

Everything grows in a **well-rounded** manner.

The Specificity Graph keeps **trending upward**.

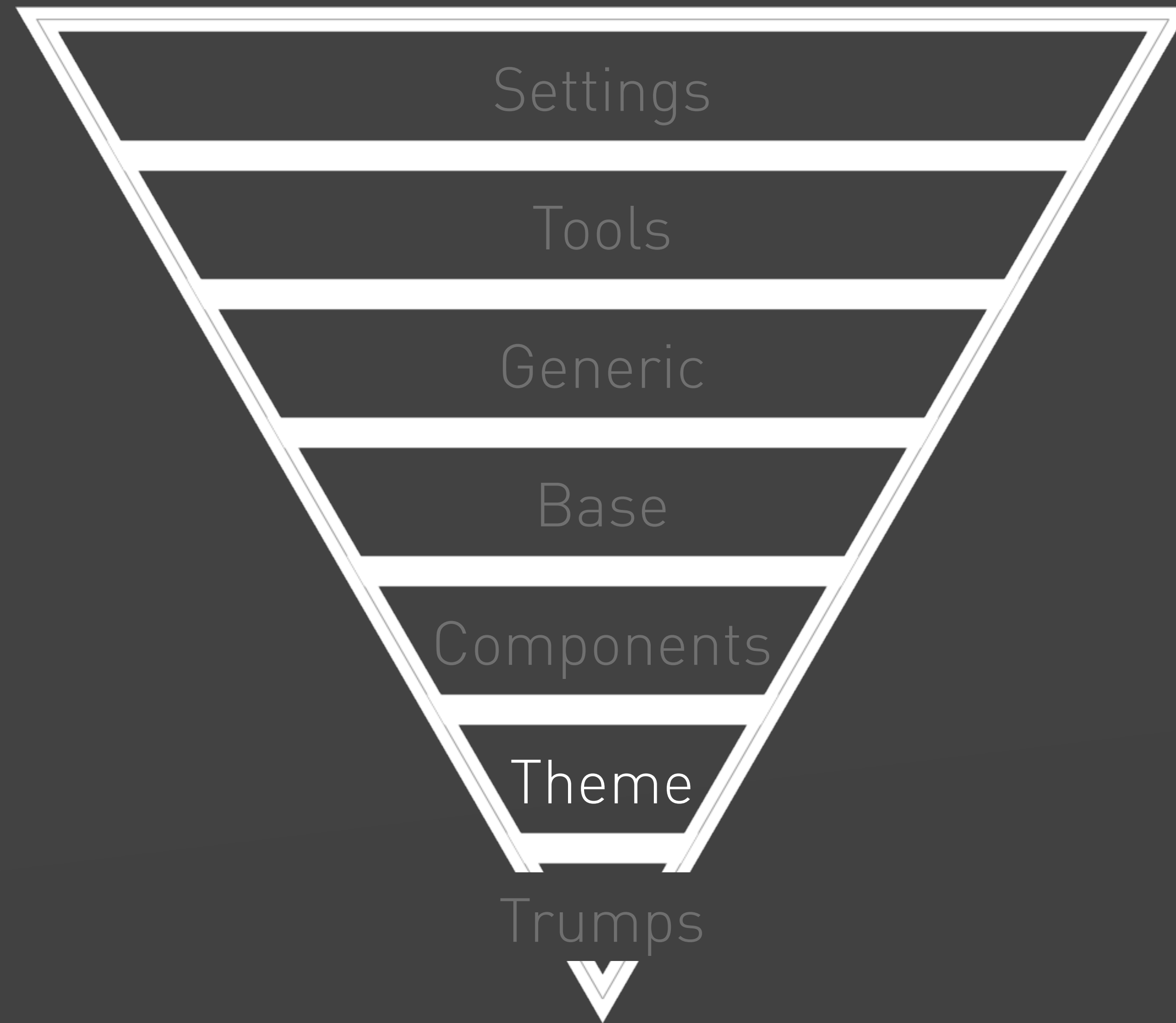
Scaling the architecture

Add or remove layers if, as, and **when you need to**.

Not using a preprocessor? **Remove** the Settings and Tools layers.

Don't use OOCSS? **Remove** the Objects layer.

Need theming? Add a **Theme layer**.



Booking.com

Booking.com run a lot of **A/B tests**.

How do we **isolate temporary styles**?

Create a **Test layer** (before the Trumps layer).

Adding layers

Add layers **in the correct place**.

Specificity and explicitness of selectors should dictate this.

Honour the Specificity Graph (always trending upward).

On the filesystem?